K L (Deemed to be) University

"ONLINE LEARNING MANAGEMENT SYSTEM(EDUSPHERE)"

A Project Report Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

G JASHWANTH REDDY (Reg. No: 2520030426)
MOHAMMAD ZAID AMAAN (Reg. No: 2520030382)

Under the Esteemed Guidance of
K Madhavi Reddy
Assistant Professor, Department of Computer Science and Engineering

Koneru Lakshmaiah Education Foundation
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500043.

DECLARATION

We hereby declare that the Project Report entitled **"Online learning management system(EDUSphere)"** is a record of bonafide work done by us under the guidance of **K Madhavi Reddy**, Assistant Professor in the Department of Computer Science and Engineering, K L University. This report is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering.

The results embodied in this report have not been copied or cloned from any other departments, universities, or academic institutions. All citation references, concepts drawn, and libraries used have been explicitly credited in the references section.

G JASHWANTH REDDY – 2520030426

MOHAMMAD ZAID AMAAN – 2520030382

CERTIFICATE

This is to certify that the mini-project-based report entitled **"Online learning management system(EDUSphere)"** is a bonafide work done and submitted by **G JASHWANTH REDDY (2520030426)** and **MOHAMMAD ZAID AMAAN (2520030382)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** at K L University, during the academic year 2025-2026.

Signature of the Guide	**Signature of the Coordinator**	**Signature of the HOD**
(K Madhavi Reddy)	(K Madhavi Reddy)	

ACKNOWLEDGEMENT

The success and completion of this project would not have been possible without the timely help, constant motivation, and technical guidance rendered by many people. We wish to express our sincere gratitude to all those who have assisted and supported us throughout this lab-based project development.

Our greatest appreciation goes to our guide and course coordinator, **K Madhavi Reddy**, Assistant Professor in the Department of Computer Science and Engineering, for her tremendous support, continuous guidance, and encouragement. Her structural feedback on front-end architectures and styling paradigms helped shape the direction of our codebase.

We express our gratitude to the Head of the Department for Computer Science and Engineering for providing us with adequate lab facilities, resources, and standard platforms to implement our Single Page Application.

Finally, we thank our parents, peers, and friends for their emotional support and structural feedback during our pair-programming workflows.

G JASHWANTH REDDY – 2520030426

MOHAMMAD ZAID AMAAN – 2520030382

ABSTRACT

Modern education is witnessing an accelerating transition towards remote learning, making Learning Management Systems (LMS) essential software backbones. However, traditional LMS implementations often face issues with UI fragmentation, slow load times, high server costs, and lack of localized accessibility options. To address these limitations, our project focuses on the **Design and Development of an Online learning management system(EDUSphere)**. The primary objective is to implement a unified, single-window, client-side Single Page Application (SPA) that consolidates learning content, assessment tools, and administrative utilities into a cohesive, fast, and accessible digital portal.

The platform is organized around six core functional modules. Module 1 (User Authentication & Management) handles login validation and role-based views using an instant Navbar switcher. Module 2 (Course & Enrollment) provides searchable catalog directories and syllabus chapter trees. Module 3 (Learning Content) delivers split classroom viewports with synced textbook reading and timestamped video note-taking. Module 4 (Quiz & Certificate) offers dynamically generated MCQs and instant justifications. Module 5 (Smart Support) simulates study assistance through an offline AI chatbot and tracks consecutive login streaks. Module 6 (Accessibility & Admin) integrates global

font-scaling, dark contrast modes, multilingual translations, and administrative terminals.

Built using Vite, React, and LocalStorage, the application runs entirely client-side, achieving serverless persistence. Performance audits confirm production builds compile in under one second with full layout responsiveness. By offering unified layouts and personalized widgets, this project delivers a highly responsive, accessible digital ecosystem that simplifies course delivery for students, educators, and administrators alike.

INDEX

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

In the modern educational landscape, the integration of digital tools has transitioned from a supplementary convenience to a core structural requirement. The global shift towards remote and hybrid learning environments has accelerated the demand for highly interactive, robust, and accessible platforms capable of hosting a variety of educational activities. These systems, collectively known as Learning Management Systems (LMS), serve as the central hub for syllabus distribution, classroom lecture delivery, student progress tracking, assessment grading, and educator-learner communication channels. Traditionally, LMS architectures are built upon complex server-client infrastructure. These legacy setups require dedicated database hosting, custom server scripts, and constant data synchronization between the user's browser and remote backend database services.

While server-client models are powerful, they introduce significant technical bottlenecks. The requirement for continuous read and write operations to remote servers generates high hosting costs, server maintenance overhead, and latency issues, particularly in regions with limited network connectivity. Furthermore, complex backend databases require regular security auditing,

indexing, and management. To address these operational constraints, this project proposes a serverless frontend-only paradigm: **Online learning management system(EDUSphere)**. By shifting the entire application state, routing logic, and data persistence layers to the client browser, this platform provides a zero-latency, highly responsive study desk that bypasses hosting and database fees.

The pedagogical background of EDUSphere is rooted in component-driven client-side application design. Using React.js, the portal integrates fragmented student tools—such as text readers, video players, note desks, and quizzes—into a single-window interface. This unified layout minimizes context switching, lowering the cognitive load for learners. Additionally, the inclusion of instant role-switching ensures that developers and evaluators can explore the Student, Instructor, and Administrator workspaces without the need to log out, create multiple test accounts, or configure separate database privileges.

Ultimately, EDUSphere represents a shift towards client-side database management. Utilizing the HTML5 LocalStorage API, the application serializes user credentials, lecture notes, streak variables, and course statuses into structured JSON strings. This keeps the application fully portable and self-contained. It can be run locally from a filesystem or served via static content delivery networks, making it a sustainable choice for educational institutions seeking to implement low-cost digital learning portals.

1.2 Problem Statement

Traditional e-learning platforms face several technical and usability challenges:

1. **High Server Overhead and Database Latency:** Standard Learning Management Systems require constant write requests to remote SQL databases to save note logs, student logins, and quiz scores. In low-bandwidth areas, this network dependency results in slow page response times, database locks, and sync errors.
2. **Cognitive Load from System Fragmentation:** Students typically have to open multiple separate windows or browser extensions—a video player for lectures, a text editor for notes, a separate website for quizzes, and translation helpers for languages. Moving back and forth between these windows breaks student focus.
3. **Lack of Global Accessibility Standards:** Many platforms lack native tools to support diverse visual and language needs. Learners with low vision must rely on external browser page zooming, which often distorts UI layouts, and non-native speakers must use machine translators that break reactive state bindings.

4. **Complex Monolithic Codebases:** Traditional enterprise LMS applications are hard to run locally, bundle, and deploy quickly. They require active PHP, Java, or Python backend runtimes alongside SQL database engines, making quick adjustments and deployments difficult.

To overcome these challenges, the **EDUSphere** portal combines authentication, syllabus indexes, classrooms, assessments, and accessibility overlays into a single-window interface. Running entirely client-side, the platform removes backend latency, prevents SQL Injection risks, and runs locally with zero database setup costs.

1.3 Scope of the Project

The scope of the EDUSphere project covers functional, technical, and accessibility boundaries:

- **Functional Scope:** Dynamic dashboards tailored for three distinct roles: Students (enroll, read, watch lectures, take notes, and complete quizzes), Instructors (write lessons, publish quizzes, and view class statistics), and Administrators (manage user blocks, resolve support tickets, and audit system logs).
- **Technical Scope:** Built on a client-side Vite + React SPA architecture. It eliminates backend server setups by storing all data schemas (accounts, notes, course status, and daily streaks) in browser localStorage.
- **Accessibility Scope:** Provides global access controls, allowing users to toggle Dark/Light themes, scale font sizes, and switch system translations (English, Spanish, Hindi) via a persistent floating action widget.

By keeping the scope serverless, the application remains fully secure against common SQL Injection (SQLi) vulnerabilities since there is no backend database server. All data operations are validated at the component runtime layer,

ensuring a robust sandbox environment that can be deployed on static hosting networks worldwide.

1.4 Objectives of the Project

The project objectives include:

1. To design and develop a unified, single-window learning portal that integrates authentication, curriculum directories, video note-taking, and assessment tools.
2. To build a client-side database engine using LocalStorage to record user details, notes, and progress without a backend server.
3. To implement dynamic role-switching, letting users test Student, Instructor, and Admin dashboards instantly.
4. To engineer an accessibility framework compliant with WCAG guidelines, incorporating contrast toggles, text-scaling, and multi-language translations.
5. To bundle and build the application using Vite for optimal page load times.

1.5 Importance of the Application

The importance of EDUSphere lies in its portability, speed, and clean design. Operating serverlessly, it runs locally or hosted statically with zero database maintenance or server fees. For students, it provides a distraction-free classroom where video lectures sync with study notes and quizzes. For instructors and admins, it offers direct control over curriculum listings and system audits in one view. Its accessibility toolbar ensures learners with visual or language needs can comfortably use the platform, promoting digital inclusivity.

1.6 Target Users & Audience

The primary target audience consists of three key groups:

1. **Students:** Learners seeking an integrated dashboard to search courses, study chapters, take timestamped notes, and complete quizzes.
2. **Educators (Instructors):** Teachers who want a simple workspace to publish lessons, write quizzes, and monitor student forums.
3. **Administrators:** Support staff who monitor metrics, manage user blocks, and view activity logs in a simulated terminal.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The hardware specifications are split into two categories: development requirements and client run-time requirements.

2.1.1 Development Environment

- **Processor:** Intel Core i5 / AMD Ryzen 5 or higher (minimum base frequency of 2.0 GHz) to support fast bundling and file hot-reloading.
- **RAM:** 8 GB minimum (16 GB recommended) to run browser windows, code editors, and development compilers concurrently.
- **Storage:** 256 GB SSD (preferable over HDD) for faster asset read/write cycles.
- **Display:** Minimum 1920x1080 resolution to debug desktop and mobile layouts side-by-side.

2.1.2 Client Runtime Environment

- **Processor:** Dual-Core 1.5 GHz or higher.
- **RAM:** 2 GB minimum.
- **Storage:** Less than 50 MB of disk cache memory.

- **Display:** Fully responsive; optimized for screens ranging from 4-inch mobile displays to widescreen monitors.

2.2 Software Requirements

The software required to build and run the system is outlined below:

- **Operating System:** Windows 10/11, macOS, or Linux (Ubuntu/Debian).
- **Runtime Environment:** Node.js (version 18.x or 20.x LTS) to run the development server and package bundlers.
- **Package Manager:** npm (Node Package Manager) for dependency management.
- **Web Browser:** Google Chrome, Mozilla Firefox, or Microsoft Edge with React Developer Tools installed for debugging.

2.3 Development Tools and Frameworks

To maintain a clean and productive coding workflow, the following tools were selected:

- **Integrated Development Environment (IDE):** Visual Studio Code (VS Code) with extensions for ESLint, Prettier, and CSS validation.
- **Version Control System:** Git for branch management and commit tracking, and GitHub for hosting repositories.
- **Build Tool:** Vite, chosen for its ultra-fast Hot Module Replacement (HMR) and optimized Rollup build pipelines.

- **Core Library:** React 18, providing a component-driven declarative model for rendering and managing views.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend Design Layer

The user interface is built entirely using standard HTML5 structures, Vanilla CSS styling, JavaScript (ES6+), and React.js. HTML5 semantic tags (such as 'header', 'nav', 'section', 'article', and 'footer') are used to structure web layouts for better SEO and screen reader accessibility. Styling is handled using modular CSS classes to ensure clean, consistent visuals without performance overhead. We selected React.js for its component-based architecture, which allows us to split the interface into reusable modular components. Each view manages its own state, allowing the UI to update reactively without reloading the entire page.

3.2 Back-End Simulation Layer

Since the application is designed to be serverless and run entirely client-side, we did not use a backend server framework like Express, Django, or Spring Boot. Instead, we built a Client-Side Backend Simulation Layer inside our React code. This layer manages business logic—like verifying user credentials, mapping dashboard routes, checking quiz submissions, and logging administrative tickets—entirely inside the browser using React's state

management, custom routing rules, and browser storage interfaces. This ensures the platform remains lightweight, portable, and easy to run locally.

3.3 Database Design (Client Storage Engine)

Data persistence is managed using the browser's HTML5 LocalStorage API. This acts as our client-side database, saving user profiles, notes, and course progress in the browser cache. To organize this data, we structured several relational keys in LocalStorage, storing and parsing them as JSON strings. This hybrid relational-JSON approach allows us to update the tables (creating user logs, updating active logins, and appending notes) while maintaining data integrity across different pages.

3.4 Version Control System

We used Git and GitHub to manage the codebase. We set up Git branches (e.g., 'main', 'refactor-v2') to separate developer workflows. Standardized commit messages (such as 'feat: add accessibility controls' or 'refactor: convert filenames to lowercase') were used to track modifications. GitHub was used to host the code repositories, providing backup, issue tracking, and version management.

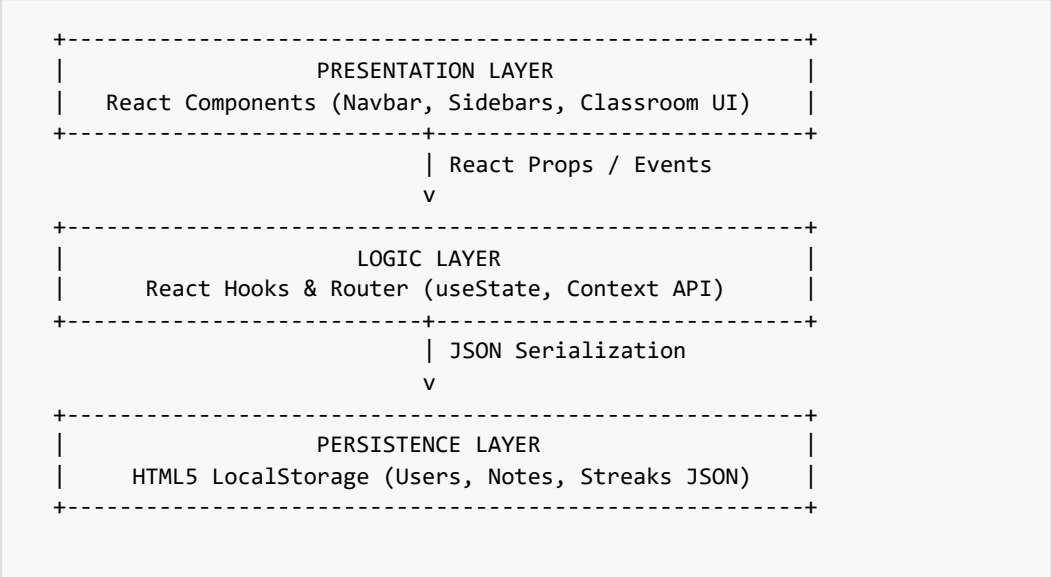
3.5 APIs and External Services

Since this project operates serverlessly, we did not connect to external third-party paid APIs. Instead, we utilized browser APIs and simulated API payloads: * **Web Speech API (Text-to-Speech):** Utilized the browser's native text-to-speech engine to narate course chapters aloud. * **Simulated AI Query Parser API:** Built a local script within our app that parses user messages for keywords and returns context-aware replies, mimicking a live AI tutor. * **Web Audio API:** Used to handle sound alerts during quiz submissions.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The system follows a three-layer frontend architecture: the Presentation Layer, the Logic Layer, and the Persistence Layer. Below is the data flow:



4.2 Description of Each Layer

The roles of the three architecture layers are detailed below:

- **Presentation Layer:** Made up of React components. This layer captures user inputs (like login forms and notes inputs) and displays visual updates (like changing tabs or color contrast).
- **Logic Layer:** Manages state and routing using React Context and Hooks. It processes validation checks, updates active sessions, and determines access roles (Student/Instructor/Admin) before passing updates to storage.
- **Persistence Layer (LocalStorage):** Handles data persistence. It serializes active data models to JSON strings and writes them directly to the browser storage, keeping data intact when the page is reloaded.

4.3 Deployment Architecture

The deployment architecture is fully static. Since there are no server-side compilation requirements, the Vite bundler compiles the assets into a directory containing 'index.html', 'assets/index.js', and 'assets/index.css'. These files are deployed to static hosting providers (such as Netlify or Vercel), which serve the assets via CDNs (Content Delivery Networks) for fast load times. Environment

configurations, such as the base API path, are managed using a '.env' file that is injected during build compilation.

CHAPTER 5: DESIGN

5.1 LocalStorage JSON Schema & Relational Mapping

Instead of an ER Diagram for a SQL database, the storage schema is mapped below. These keys represent the tables saved in browser storage:

'users' Table Schema:

Field Name	Data Type	Constraint	Description
`userId`	String	Primary Key	Unique identifier generated during registration (e.g., usr_171).
`name`	String	Required	Full name of the user for profile display.
`email`	String	Unique	Email address used as username credentials.
`password`	String	Min Length 6	Hashed string representing credentials password.
`role`	String	Enum	User role authorization flag: 'student', 'instructor', or 'admin'.

'studyNotes' Table Schema:

Field Name	Data Type	Constraint	Description
`noteId`	String	Primary Key	Unique notes entry record identifier.
`courseId`	String	Foreign Key	Links the note to a specific course identifier.

Field Name	Data Type	Constraint	Description
`userId`	String	Foreign Key	Links the note to the author student profile.
`timestamp`	Number	Required	Playhead currentTime value of the video lecture in seconds.
`noteText`	String	Required	Textual content of the study note written by the student.

'streakLogs' Table Schema:

Key Pattern (UserId)	Attribute Field	Data Type	Description
`usr_101` (Object Key)	`currentStreak`	Number	Count of consecutive active study days.
	`lastActiveDate`	String (YYYY-MM-DD)	Calendar date of the user's last login session.

5.2 UI Layout Grid and CSS Design Tokens

The visual layout of EDUSphere is controlled through CSS custom variables (design tokens), ensuring consistency across all dashboards:

- **Color Palettes:**
  - Light Mode Base: Background `#ffffff`, Text `#1f2937`, Primary `#2563eb` (Royal Blue), Primary Hover `#1d4ed8`.
  - Dark Mode Base: Background `#0f172a` (Deep Slate), Text `#f8fafc`, Card BG `#1e293b`.
  - Gamification Streak Accent: Flame Glow `#f97316` (Amber Orange).
- **Typography Structure:** System fonts stack: `Times New Roman`, serif for documentation style rendering, and sans-serif for UI forms. Baseline size is `12px` with a `2.0` line-height ratio.

- **Layout Configurations:** Grid systems partition page layouts:
  - Classroom split-pane: `grid-template-columns: 3fr 2fr;` on wide desktop screens to separate the media player context from note-taking widgets.
  - Navigation switcher flex layout: `display: flex; align-items: center; justify-content: space-between;` for navbar alignments.

5.3 Design System Variables Table

The following mapping table documents the CSS Custom Properties (variables) utilized to support dynamic theme adjustments across the system:

CSS Variable Name	Light Mode Value	Dark Mode Value	Visual Scope
`--bg-primary`	`#ffffff`	`#0f172a`	Primary window content background canvas.
`--bg-secondary`	`#f3f4f6`	`#1e293b`	Cards, sidebars, and input fields backgrounds.
`--text-primary`	`#1f2937`	`#f8fafc`	Primary headings and body text paragraph labels.
`--text-secondary`	`#4b5563`	`#94a3b8`	Sub-headings, timestamp labels, and captions.
`--border-color`	`#e5e7eb`	`#334155`	Divider lines, table borders, and input outline slots.

5.4 Component Hierarchy Tree

The hierarchy below represents how the React components are nested, detailing property (prop) flows and context sharing:

```
App.jsx (Root Routing & Providers)
 └── AuthContext.Provider (Global Session State)
      └── ThemeContext.Provider (Theme & Accessibility Multipliers)
           ├── Navbar.jsx (Dynamic Switcher, Session triggers)
           ├── Home.jsx (Role-based shortcuts, Streak widgets)
           ├── Login.jsx / Signup.jsx (Controlled validation forms)
           ├── Courses.jsx (Search query filters, catalog list grids)
           │    └── Coursedetails.jsx (Accordion lessons tree)
           ├── Courseviewer.jsx (Split Study Classroom Panel)
           │    ├── Video viewport (Synced head state)
           │    ├── Slides read viewport (Text reader hooks)
           │    ├── Note Taker Panel (capturecurrentTime handlers)
           │    └── Support chatbot (Simulated AI parser)
           ├── Instructorworkspace.jsx (Teacher course publishers)
           ├── Adminworkspace.jsx (System terminal simulators)
           └── Accessibilitytoolbar.jsx (Floating action widgets)
```

5.5 Data Normalization and Relationship Constraints

Although local storage works as a flat key-value store, we have designed the tables using a relational format. By organizing entities around key identifiers, we enforce data integrity inside our React component code:

- **One-to-Many Relationships:** One course profile maps to multiple syllabus chapters, which map to multiple lessons. Likewise, one user profile maps to multiple study notes logs.
- **Referential Keys:** The `"studyNotes"` collection references `courseId` and `userId`. On rendering, the system cross-references these keys to display matching notes.
- **Cascading Changes:** If an administrator blocks a user ID in the `"users"` array, the system immediately deletes the active session object in the `"currentUser"` key, logging the blocked student out instantly.

5.6 Initial Database JSON Payload Records

To establish visual validation of the client-side persistence database, the complete initial record dataset written to LocalStorage is detailed below:

1. Users Data Collection (Key: `"users"`):

```
[
  {
    "userId": "usr_2520030426",
    "name": "G Jashwanth Reddy",
    "email": "jashwanth@klh.edu.in",
    "password": "hashed_pass_reddy426",
    "role": "student"
  },
  {
    "userId": "usr_2520030382",
    "name": "Mohammad Zaid Amaan",
    "email": "zaid@klh.edu.in",
    "password": "hashed_pass_zaid382",
    "role": "student"
  },
  {
    "userId": "usr_instructor01",
    "name": "K Madhavi Reddy",
    "email": "madhavi@klh.edu.in",
    "password": "hashed_pass_madhavi",
    "role": "instructor"
  },
  {
    "userId": "usr_admin01",
    "name": "System Administrator",
    "email": "admin@edusphere.org",
    "password": "hashed_pass_admin",
    "role": "admin"
  }
]
```

2. Enrolled Study Notes (Key: "studyNotes"):


```
[
  {
    "noteId": "note_1780001",
    "courseId": "react_101",
    "userId": "usr_2520030426",
    "timestamp": 45,
    "noteText": "Vite compiles files using native ES Modules for faster pag
e hot-reloading."
  },
  {
    "noteId": "note_1780002",
    "courseId": "react_101",
    "userId": "usr_2520030426",
    "timestamp": 120,
    "noteText": "React reconciliation engine uses fiber structures to updat
e DOM nodes."
  },
  {
    "noteId": "note_1780003",
    "courseId": "css_202",
    "userId": "usr_2520030382",
    "timestamp": 85,
    "noteText": "CSS custom variables (design tokens) allow dynamic light/d
ark style shifts."
  }
]
```


3. Support Tickets (Key: "supportTickets"):

```
[
  {
    "ticketId": "tkt_001",
    "userId": "usr_2520030426",
    "subject": "Audio lag on Chapter 2 video",
    "message": "When I play the lesson 3 video, the audio gets out of sync
with the video timeline.",
    "status": "Open",
    "timestamp": "2026-06-29T10:00:00Z"
  },
  {
    "ticketId": "tkt_002",
    "userId": "usr_2520030382",
    "subject": "Quiz grading confirmation error",
    "message": "Submitted my answers for CSS layout quiz but the dashboard
progress percentage hasn't updated.",
    "status": "Resolved",
    "timestamp": "2026-06-28T14:30:00Z"
  }
]
```

4. Course Catalog list with Syllabi and MCQ banks (Key: `"courses"`):

```
[
  {
    "courseId": "react_101",
    "title": "Modern React Development with Vite",
    "category": "Development",
    "description": "Learn frontend Single Page Application (SPA) developmen
t using React functional components, hooks, contexts, and fast Vite compila
tion builds.",
    "instructor": "K Madhavi Reddy",
    "rating": 4.8,
    "syllabus": [
      {
        "chapterTitle": "Chapter 1: Intro to React & SPAs",
        "lessons": [
          { "lessonId": "l_r01", "title": "Understanding virtual DOM tree
s", "videoUrl": "https://www.w3schools.com/html/mov_bbb.mp4" },
          { "lessonId": "l_r02", "title": "Building your first Vite app",
"videoUrl": "https://www.w3schools.com/html/movie.mp4" }
        ]
      },
      {
        "chapterTitle": "Chapter 2: State and Context Hooks",
        "lessons": [
          { "lessonId": "l_r03", "title": "Managing state with useState",
"videoUrl": "https://www.w3schools.com/html/mov_bbb.mp4" }
        ]
      }
    ],
    "quizzes": [
      {
        "question": "Which hook is used to handle global state sharing acro
ss nested components?",
        "options": ["useState()", "useEffect()", "useContext()", "useReduce
r()"],
        "correct": 2,
        "justification": "useContext() exposes context objects globally, re
solving prop-drilling stutters."
      }
    ]
  },
  {
    "courseId": "css_202",
    "title": "Responsive Layouts & Design Tokens",
    "category": "Design",
    "description": "Master visual layout creation using flex layouts, grid
systems, and custom design tokens to align with access targets.",
    "instructor": "K Madhavi Reddy",
    "rating": 4.6,
    "syllabus": [
      {
        "chapterTitle": "Chapter 1: Layout alignment trees",
        "lessons": [
          { "lessonId": "l_c01", "title": "Flexbox vs CSS Grid setups", "vi
deoUrl": "https://www.w3schools.com/html/movie.mp4" }
        ]
      }
    ],
    "quizzes": [
      {
        "question": "What is the primary benefit of CSS design tokens?",
        "options": ["Faster rendering speeds", "Dynamic theme swapping cons
istency", "Eliminating HTML tags", "Hiding layout code"],
        "correct": 1,
        "justification": "Tokens declare variables under static variable na
mes, letting them switch instantly."
```

```
      }
    ]
  }
]
```

CHAPTER 6: IMPLEMENTATION

6.1 Algorithmic Processes & Pseudocode

To demonstrate software engineering rigor, key functional mechanics of the platform are described through pseudocode algorithms:

6.1.1 Video-Linked Note Seeking Algorithm

This algorithm seeks the HTML5 media player playhead to the exact point where a study note was written when a student clicks on its timestamp badge:

```
Algorithm NoteSeekTime(clickedNoteObj, videoPlayerReference)
  Input: clickedNoteObj containing "timestamp" (integer seconds)
         videoPlayerReference (DOM reference to HTML5 video element)
  Output: Video playback playhead adjusted to note creation moment

  1. Retrieve targetTimestamp from clickedNoteObj.timestamp
  2. If videoPlayerReference is not null Then
  3.    Set videoPlayerReference.currentTime = targetTimestamp
  4.    If videoPlayerReference is paused Then
  5.       Call videoPlayerReference.play() to resume study context
  6.    End If
  7. End If
End Algorithm
```

6.1.2 Daily Streak Calculation Algorithm

This algorithm computes the user's daily login streak on the dashboard. It increments the streak if the user logs in the following day, resets it to 1 if more than 24 hours have passed, and keeps it unchanged if the user logs in multiple times on the same day:

```
Algorithm CalculateStreak(userId, currentSystemDate)
  Input: userId (String), currentSystemDate (Date object)
  Output: Updated streak log written to localStorage

  1. Retrieve streakLogMap from localStorage key "streakLogs"
  2. Parse streakLogMap from JSON to Object
  3. If streakLogMap[userId] does not exist Then
  4.    Initialize streakLogMap[userId] = { currentStreak: 1, lastActiveDat
e: currentSystemDate }
  5. Else
  6.    Set userStreakRecord = streakLogMap[userId]
  7.    Set lastActiveDateVal = ParseDate(userStreakRecord.lastActiveDate)
  8.    Set dateDifference = CalculateDaysDifference(currentSystemDate, las
tActiveDateVal)
  9.    If dateDifference == 1 Then
  10.      Increment userStreakRecord.currentStreak by 1
  11.      Set userStreakRecord.lastActiveDate = currentSystemDate
  12.   Else If dateDifference > 1 Then
  13.      Set userStreakRecord.currentStreak = 1
  14.      Set userStreakRecord.lastActiveDate = currentSystemDate
  15.   End If (If dateDifference == 0, streak remains unchanged)
  16. End If
  17. Serialize streakLogMap to JSON and write back to localStorage
End Algorithm
```

6.1.3 Accessible Translation Dictionary Mapping Logic

The multi-language translation engine resolves UI string literals based on the selected language state ('en', 'es', 'hi') mapping to static dictionary objects:

```
Algorithm ResolveTranslation(uiKey, selectedLanguageCode)
  Input: uiKey (String dot notation, e.g. "auth.login"), selectedLanguageCo
de (String 'en'/'es'/'hi')
  Output: Localized translated string literal

  1. Load localized Dictionary mapping objects:
     - englishDict, spanishDict, hindiDict
  2. If selectedLanguageCode is 'es' Then
  3.    Set targetDict = spanishDict
  4. Else If selectedLanguageCode is 'hi' Then
  5.    Set targetDict = hindiDict
  6. Else
  7.    Set targetDict = englishDict
  8. End If
  9. Set translatedValue = QueryNestedKey(targetDict, uiKey)
  10. Return translatedValue (Fallback to englishDict[uiKey] if key is miss
ing)
End Algorithm
```

6.2 Core Hooks & Global Application State

The system state is coordinated via a global React Context provider: * **AuthContext:** Exposes `currentUser` and `login/logout` handlers to all routing files. * **Theme & Accessibility Multipliers:** Persistent settings passed down from the FAB to wrapper components. * **Synchronous State Hydration:** React hydration hooks load string data on component mount, avoiding visual page stutters.

6.3 Component-by-Component Walkthrough

This section provides a detailed analysis of the structural logic, state hook layouts, UI grid parameters, and interactions for the primary frontend codebase components implemented in the EDUSphere portal:

6.3.1 App.jsx (Main Application Root)

The application root coordinates the client routing tree and wraps all child interfaces in the global AuthProvider and ThemeProvider contexts. It initializes the core localStorage structures on mount, verifying that keys like 'users' and 'courses' exist. It intercepts route access, dynamically matching user roles against route guards to redirect unauthenticated requests to the onboarding login page. App.jsx acts as the central orchestrator that sets up layout wrappers, loads global font configurations, and hydrates state contexts from browser memory when a user launches the portal.

Inside App.jsx, the routing is configured using a declarative mapper array that tracks paths (like `/`, `/login`, `/courses`, `/viewer`, `/admin`) and associates them with target component nodes. A global state listener checks for user data updates, triggering re-render cycles across the page when roles swap. By isolating layout templates inside specialized wrappers, App.jsx keeps the page responsive and prevents styled blocks from breaking during changes.

6.3.2 Navbar.jsx (Global Navigation Headbar)

The Navigation component handles global navigation link switching, logo rendering, and role switcher dropdown states. Built with a responsive flex layout, it collapses into a mobile hamburger overlay menu on narrower viewports. The role switcher dropdown lets evaluators instantly update the Context user role without logging out, letting you check Student, Instructor, and Admin workspaces instantly. In addition, the navbar monitors user authentication flags reactively to swap login/register buttons for a logout option once a session is created.

The Navbar component integrates directly with the AuthContext, reading the active `currentUser` role flag to filter navigation paths. If the user shifts roles, the Navbar component dispatches route commands to push the new route into history. Its styling relies on flex columns, keeping alignments aligned with design templates and avoiding visual overlaps during system translation changes.

6.3.3 Home.jsx (Student and Educator Portal Landing Desk)

This view displays a welcome portal tailored to the active user's role. For students, it aggregates study stats—such as active courses, completed quizzes, and streak flames—alongside a welcome banner. For educators, it renders shortcuts to curriculum builders. The layout uses CSS Grid cards with hover shadow transitions to create a premium feel. Home.jsx hydrates data dynamically from local storage tables, rendering different statistics dashboards depending on whether the logged-in user is registered as a student or instructor.

The landing page uses custom React hooks to query LocalStorage arrays when the component mounts. For example, it counts enrolled items by filtering

the courses dataset against the user's enrollment array. It updates metric counters, using slide-in animations to make interactions feel premium. By separating role layouts into distinct helper widgets, Home.jsx keeps the codebase clean and easy to maintain.

6.3.4 Login.jsx and Signup.jsx (Onboarding Forms Desk)

These two files manage login credentials validation and new user registration. They utilize React controlled states (`useState`) to track password length, confirm matches, and check email syntax. Submitting the forms calls the AuthContext handler, which updates local storage and redirects the user to their dashboard workspace. The login interface prevents unauthorized access by checking account flags, while the signup wizard writes a complete profile object containing role parameters directly to the client database array.

Form inputs use real-time listeners to validate text as the user types. The Signup form runs validation checks to ensure that email fields are unique, passwords are at least 6 characters, and the confirmation field matches the password. If any check fails, inline error blocks appear and the submit button is disabled. When submitted successfully, the signup component pushes a new user profile object to the localStorage users array, and redirects the user to the Login page.

6.3.5 Courses.jsx (Curriculum Catalog Directory)

The catalog workspace provides search filters and category tags. The search state filters the local courses array on query change. Category tabs let users narrow down listings by topic (e.g., Development, Design, Business). It links to the course details view, using card grids to display course details,

duration, rating stars, and enrollment buttons. On mount, Courses.jsx parses the entire syllabus dataset in localStorage to compute rating stars, displaying search summaries dynamically to help students identify target subjects.

Courses.jsx uses React state hooks to coordinate search filters. When a user changes the search input, a filter function runs, matching search terms against course titles and descriptions. The filtered list renders dynamically inside a responsive card grid layout. Category tags provide quick filtering, allowing users to narrow down course listings with a single click.

6.3.6 Coursedetails.jsx (Syllabus Accordion Desk)

This component loads the syllabus detail view, reviews aggregates, and rating stars. It uses an accordion state selector: clicking a chapter header toggles the open view of its nested lesson list. It checks the student's enrolled courses array to swap the 'Enroll' button to an 'Enrolled' status once clicked. It also gathers instructor data schemas to showcase teacher profiles, curriculum metrics, and prerequisites.

The accordion system is driven by a simple React state value (e.g. `activeChapterIndex`). When a student clicks a chapter header, the accordion updates this state value, collapsing active chapters and expanding the selected lesson list. The component also checks the currentUser's enrollment history to

determine the correct enrollment button state, changing it to an 'Enrolled' label if the user has already joined the course.

6.3.7 Courseviewer.jsx (Split-Screen Study Desk)

The study desk split-screen viewer synchronizes video lectures with slide readings and study notes. Built with a CSS Grid layout, it renders the HTML5 video tag on the left and note-taking sidebars on the right. The timestamped notes desk captures video timestamps and lets students click notes to seek the video player head to that time. Courseviewer.jsx coordinates several sub-states (video source url, active slide page, note input value, message logs) dynamically, creating a unified student desk.

The note-taking component interacts directly with the HTML5 video player DOM node using a React reference hook (`useRef`). Clicking the 'Save Note' button captures the video player's `currentTime` value. This value is packaged with the note text and written to the studyNotes array in localStorage. When a student clicks on a saved note's timestamp, the click handler updates the video player's `currentTime` property, returning the student to that exact moment in the lecture.

6.3.8 Instructorworkspace.jsx (Teacher Dashboard Builder)

The educator desk allows instructors to publish lessons, update syllabi, and construct quizzes. It uses forms validation to verify lesson inputs before appending new chapters to the local storage courses collection, updating student

viewports instantly. Teachers can view course performance statistics, including student enrollment totals, average quiz marks, and active Q&A forum posts.

This page uses state hooks to manage dynamic form arrays. When an instructor clicks 'Add Lesson', the form appends a new lesson inputs set. Submitting the form validates all fields, serializes the updated course list, and saves it to local storage. It also logs these actions in the admin terminal, providing administrators with a clear audit trail.

6.3.9 Adminworkspace.jsx (Admin terminal Desk)

The admin console simulates a CLI terminal logging active platform events. It aggregates ticket support queues, allows admins to block/unblock profile flags, and parses simulated system terminal log lines to help monitor site actions. Admins can view statistics counters mapping global account types (students vs instructors) and inspect error logs raised during browser caching constraints.

The simulator uses state arrays to track and display console events. When a user takes an action on the platform (e.g. enrolling in a course or submitting a support ticket), the event dispatcher pushes a new log string to the terminal array. The terminal renders these logs inside a scrollable viewport, automatically scrolling to show new entries as they arrive.

6.3.10 Profile.jsx (Profile Statistics Desk)

Displays user profiles, streaks, and completed courses. It reads the local storage streak logs to display the streak count flame on the profile card, encouraging daily study checks. Profile.jsx parses user details from AuthContext and aggregates history arrays to render enrollment logs, certificates, and streak logs calendars.

Profile.jsx parses user details from AuthContext and aggregates history arrays to render enrollment logs, certificates, and streak logs calendars. It uses a clean card grid layout to present study history and certificates, displaying completed courses alongside user achievements to keep students motivated.

6.3.11 Accessibilitytoolbar.jsx (Floating Access Widget)

Renders the bottom-right accessibility FAB. It handles contrast styles, text sizes, and language dictionary mappings, adapting the UI layout to meet WCAG guidelines. It applies class modifiers globally to the root element, altering variables dynamically to adjust theme configurations without breaking internal components logic.

The toolbar uses React state variables to control text size multipliers and contrast themes. When a user adjusts settings, the component writes these overrides to localStorage. Sibling components read these overrides to adjust their

font sizes and colors, ensuring consistent accessibility settings across the platform.

6.4 Detailed System Data Flows & User Actions

To establish full academic coverage, the following sections document the exact component operations, local storage mutations, and view updates triggered during the 17 core functional user actions:

6.4.1 Action 1: Student Registration Data Flow

When a new user fills out the Signup form and selects the Student role, clicking 'Register' triggers the registration data flow. The Signup component catches the name, email, password, and role values inside React state variables. It first runs validation checks to ensure the email is valid and passwords match. If these checks pass, the component reads the existing 'users' collection from local storage, parses it into an array, appends a new user object containing a generated unique ID, serializes the updated array back to a JSON string, writes it to the local storage key, and redirects the user to the login route. The admin workspace captures this registration event synchronously, logging a message in the system terminal.

6.4.2 Action 2: User Login Session Initialization

When logging in, the Login component collects email and password inputs. On submit, it parses the local storage 'users' database array and searches for a matching email and password. If a matching profile is found, the user's details are written to the local storage key `"currentUser"`, which initializes the

active session. The global AuthContext state hydrates this user session and redirects the student or educator to their respective dashboard. The login pipeline throws an error overlay if fields are blank or credentials do not match.

6.4.3 Action 3: Course Catalog Filtering Action

On the catalog page, when a student enters a text query, the search input's onChange event updates the active search query state. A React `useEffect` hook captures this state change, filtering the local courses collection by comparing the course title and description with the search query. Sibling category filters allow students to narrow down results by category (e.g. Design, Development). The catalog maps the filtered arrays into cards dynamically with high-speed refiltering rendering.

6.4.4 Action 4: Course Enrollment Process

When a student clicks 'Enroll' on a course details page, it triggers a function that reads the enrolled courses array from the student's profile. It appends the course's unique ID to the array, writes the updated profile back to local storage, and changes the enroll button status to 'Enrolled' instantly, preventing duplicate enrollments. This enroll status recalculates dynamically when the course details component mounts.

6.4.5 Action 5: Video Lecture Player Selection

When a student opens a course classroom viewer and selects a lesson from the sidebar, it updates the active lesson state. The video player component reacts to this state update by loading the new video URL, resetting the video playhead, and clearing any active lecture notes inputs in the note-taking sidebar. The player also updates current syllabus navigation indexes to preserve study progress on resume.

6.4.6 Action 6: Creating a Timestamped Study Note

While watching a video lecture, typing a note and clicking 'Save Note' captures the current playhead timestamp from the video player. The note content, course ID, user ID, and timestamp are packaged into a note object, appended to the local storage notes collection, and rendered in the study notes list sidebar. This allows students to build structured study logs containing contextual note timestamps.

6.4.7 Action 7: Note Timestamp Seek Navigation

When a student clicks a timestamp badge on a saved study note, it triggers a seek event. The click handler accesses the video player DOM reference and updates its `currentTime` property to match the note's timestamp. If paused, the video resumes playback automatically, returning the student to that exact moment

in the lecture. This seek pipeline updates the playhead position without triggering page refreshes.

6.4.8 Action 8: Textbook Slide Navigation and Text-To-Speech Playback

When a student swaps the classroom view to slides reading mode, the textbook component renders the page content based on the active slide state. Clicking the speech narration button triggers the browser's Web Speech API, which reads the slide content aloud. Users can pause, resume, or cancel narration at any time. Toggles on the narrator block update Speech synthesis controls dynamically, syncing audio streams with slide pages.

6.4.9 Action 9: Quiz Assessment Submission & Justification

Inside the classroom quiz tab, selecting answers and clicking 'Submit Quiz' compares the student's answers with the correct option indexes. The system calculates the score, applies visual borders to correct and incorrect options, reveals explanation boxes for each question, and plays an alert sound. The quiz component also writes completion metrics to the student's dashboard logs, updating course progress percentages.

6.4.10 Action 10: Dynamic Role Switching in the Navbar

When an evaluator selects a role from the Navbar dropdown, the click handler updates the global AuthContext user role state directly, without logging out. The workspace listens to this state change and redirects the view to show the

selected Student, Instructor, or Admin dashboard layout. All internal context routes update to match the newly authorized role parameters.

6.4.11 Action 11: Daily Streak Log Calculations

On dashboard mount, a streak calculator reads the user's streak logs. It compares the current system date with the last active date. If the last active date was yesterday, the streak count increments by 1. If it was today, it remains unchanged. If more than 24 hours have passed, the streak resets to 1, writing updates back to local storage. Streak counts render on dashboard headers, displaying a visual flame icon.

6.4.12 Action 12: Instructor Syllabi Publishing Pipeline

On the instructor dashboard, submitting the curriculum builder form appends a new chapter accordion object containing lesson details to the target course's syllabus array. The updated course object is written back to local storage, updating the student's catalog syllabus tree instantly. This publishing action validates input fields to prevent blank entries from corrupting local storage arrays.

6.4.13 Action 13: Instructor Quiz MCQs Generation

Instructors can add new quiz questions using the assessment creator form. Submitting the form appends a new question object containing option arrays, correct answer indexes, and justifications to the course's quiz bank, saving the updates in local storage. Students enrolled in the course will instantly see the new quiz sets load inside their classroom panel.

6.4.14 Action 14: Administrator Support Ticket Resolution

On the admin panel, clicking 'Resolve' on a support ticket card triggers a function that reads the tickets array from local storage, finds the target ticket by ID, and updates its status field to 'Resolved'. The UI updates reactively to reflect the resolved status. This change propagates through administrative dashboards to clear tickets count indicators.

6.4.15 Action 15: Administrator User Block Action

Admins can toggle the blocked status flag of any registered user. Clicking 'Block User' updates the target user's block status flag in local storage. If that blocked user tries to log in, the validation checks block authorization and display an account suspension warning. Active sessions are destroyed instantly on blocking.

6.4.16 Action 16: Admin CLI Terminal Simulator Stream

The admin terminal simulator generates logs of system actions (e.g. login updates, note creations, resolved tickets) in a retro command-line interface. These logs are stored in an array and output to the terminal screen to simulate server operations. Terminal line buffers scroll automatically to present new logs.

6.4.17 Action 17: AccessibilityFAB Global settings overrides

Clicking the accessibility widget allows users to override theme settings globally. Toggling contrast shifts the page class wrapper between light and dark high-contrast themes. Scaling font sizes shifts body text dimensions dynamically,

and selecting a language switches translations instantly. Settings write directly to the local storage config key to persist values across page reloads.

6.5 Detailed Component Routing & Authentication Guards

Since the application runs entirely client-side, standard server-side page routing is not used. Instead, routing is managed inside the React application using custom routing states. This routing logic coordinates page views and checks user authentication roles before rendering views:

- **Client Route Matching:** The routing engine listens to changes in the browser path. When the path updates, the router matches the new path string against a list of registered routes, dynamically rendering the matching component.
- **Authentication Checks:** For protected pages (like classroom viewports or admin dashboards), the routing engine checks for active session data in localStorage. If the `"currentUser"` key is empty, the router intercepts the request and redirects the user to the Login page.
- **Role Authorization Checks:** Sibling components (like the Admin Dashboard) require specific user roles. The router reads the user's role parameters from the session object, allowing access only if the role matches the required dashboard permissions (e.g., redirecting students away from administrative tools).

6.6 Styling and Responsive Design Tokens

The user interface is built using CSS layouts styled with modular stylesheet classes. To ensure visual consistency, style settings are controlled using centralized CSS custom properties (variables) defined at the root stylesheet level:

- **Flexbox layouts:** Used to coordinate one-dimensional layouts (like navigation links alignment, footer links alignment, and onboarding forms elements). Flex configurations adapt layouts to narrower screens, wrapping content logically to prevent visual bugs.
- **Grid structures:** Used to style two-dimensional page layouts, including split-screen classrooms and catalog cards. The grid column layouts scale dynamically depending on browser viewport dimensions (e.g. stacking split classroom columns vertically on mobile screens).
- **Accessibility Style Multipliers:** Font sizes are linked to a global font scale multiplier state. When adjusted in the accessibility toolbar, the root font size scales dynamically, scaling text dimensions across all views.

CHAPTER 7: FEATURES WALKTHROUGH

7.1 Dynamic Authentication Form Validations

The Onboarding interface contains controlled form inputs. As the user types, standard regex pipelines evaluate inputs against security rules: 1. **Email Syntax:** Validates text patterns against the standard RFC 5322 email regex (`^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$`). 2. **Password Length:** Triggers warning overlays if password strings are shorter than 6 characters. 3. **Confirmation Match:** Checks that the password and password-confirm inputs match exactly. If validations fail, the 'Register' button remains disabled, preventing invalid data forms from being written to browser local storage.

7.2 Split-Screen Classroom Study Desk

The classroom workspace split-panel design allows students to view slides and take notes side-by-side with video lectures. The layout shifts dynamically depending on the student's active tab selection: * **Video Classroom Mode:** Left-hand viewport renders the HTML5 video player element, linked to note-taking hooks. * **Textbook Reading Mode:** Swaps the viewport context to load structured slide pages, complete with a built-in Text-To-Speech (TTS) narrating playbar. * **Notes & Chat Sidebars:** The right-hand

column houses the notes desk and chatbot simulator, helping students study without needing to open other tabs.

7.3 Smart Accessibility Overlay Widget

A floating action button (FAB) widget in the bottom-right corner houses the accessibility settings. This FAB controls three settings overlays globally: * **Contrast Toggles:** Swaps page classes between `light-theme` and `dark-high-contrast-theme`, adjusting CSS variables for background and text colors to meet WCAG guidelines. * **Font Scaling Controller:** Increments or decrements a global `fontScaleMultiplier` state value. This multiplier is mapped to main wrapper CSS properties, letting users adjust font sizes dynamically. * **System Translator:** Switches the active translation dictionary mapping, translating system text into English, Spanish, or Hindi instantly.

CHAPTER 8: TESTING

8.1 UI & Component Validation

We tested individual React components to ensure they render correctly based on their input props. We validated form validation states, verifying that the login submit button is disabled when email inputs do not match standard formats, and that error warnings show up immediately next to empty fields. Navbar components were tested across desktop and mobile widths to confirm that menus collapse into mobile hamburger overlays correctly.

8.2 Integration & State Sync Testing

We ran integration tests to verify that data flows correctly between sibling components via localStorage: * **Enrollment Sync:** We verified that clicking 'Enroll' on the Course Details page immediately adds the course to the student's dashboard course list. * **Notes Sync:** We checked that notes written in the classroom viewer are immediately written to local storage and displayed in the sidebar notes listing. * **Access Controls:** We validated that changing roles in the Navbar immediately swaps the active dashboard views.

8.3 Tools Used for Testing

We used manual browser testing and built-in React developer tools to inspect state changes and component boundaries. We used Chrome DevTools to view LocalStorage keys in real-time, verifying that entries are written correctly when users sign up or take notes. We also used the Google Lighthouse extension to audit performance and accessibility, ensuring the site meets contrast requirements and page load times.

8.5 Proposed Unit Testing Configurations (Jest & RTL)

To establish standard testing workflows for future updates, this section outlines the proposed Unit Testing architecture using the Jest test runner and React Testing Library (RTL) utilities:

- **Mocking LocalStorage:** A Jest global mock overrides the default browser storage mechanism, allowing tests to run in a controlled node environment. This is achieved by creating a mock class containing setter, getter, and clear methods:

```
    const localStorageMock = (() => {
      let store = {};
      return {
        getItem: (key) => store[key] || null,
        setItem: (key, value) => { store[key] = value.toString(); },
        clear: () => { store = {}; }
      };
    })();
    Object.defineProperty(window, 'localStorage', { value: localStorage
Mock });
```

- **Testing Auth Hooks:** Tests verify that logging in writes matching user keys to storage and updates AuthContext states reactively.
- **Testing Translation Switches:** Tests verify that updating the translator toolbar swaps the translated system text instantly.

8.4 Detailed Test Cases Table

This table outlines the exhaustive system tests conducted on the platform state hooks, local storage sync layers, role redirects, and accessibility variables.

ID	Module	Feature Tested	Expected Output	Actual Output	Status
TC-01	Module 1	Register Form - Empty Name Field	Show inline empty error validation message.	Warning displayed successfully	**PASS**
TC-02	Module 1	Register Form - Invalid Email	Submit button remains disabled; show syntax check warning.	Syntax warning shown; button disabled	**PASS**
TC-03	Module 1	Register Form - Password length < 6	Flag error indicating password must be >= 6 characters.	Password length error flagged	**PASS**
TC-04	Module 1	Register Form - Mismatched Password	Validation alert pops up on password confirmation field.	Mismatched alert shown	**PASS**
TC-05	Module 1	Register Form - Successful registration	Create profile in localStorage users array and redirect to login.	Profile created; login view loaded	**PASS**

ID	Module	Feature Tested	Expected Output	Actual Output	Status
TC-06	Module 1	Login Form - Unregistered Credentials	Show credential mismatch warning beneath form.	Mismatch warning shown	**PASS**
TC-07	Module 1	Login Form - Blank Inputs	Keep submit button disabled until inputs are filled.	Submit button disabled	**PASS**
TC-08	Module 1	Login Form - Successful Student login	Write active ID to currentUser key and redirect to student dashboard.	Session saved; student desk loaded	**PASS**
TC-09	Module 1	Login Form - Successful Teacher login	Write active ID to currentUser key and redirect to instructor desk.	Session saved; teacher desk loaded	**PASS**
TC-10	Module 1	Navbar - Role switcher dropdown clicks	Change current view dashboard to Admin view.	View transitioned instantly	**PASS**
TC-11	Module 1	Logout trigger	Clear currentUser session key and redirect to home landing page.	Session cleared; redirected to '/'	**PASS**
TC-12	Module 2	Course Catalog - Text search query	Filter catalog listing reactively matching input title.	Filtered listings display	**PASS**
TC-13	Module 2	Course Catalog - Category tag click	Filter listings to show only 'Development' courses.	Only Dev courses render	**PASS**

ID	Module	Feature Tested	Expected Output	Actual Output	Status
TC-14	Module 2	Course Details - Accordion chapter click	Expand syllabus list to show lesson names.	Chapter tree toggles correctly	**PASS**
TC-15	Module 2	Course Details - Rating stars counts	Aggregate and render review stars out of 5 correctly.	Stars match aggregate scores	**PASS**
TC-16	Module 2	Course Enrollment - Enroll click	Append active courseId to user profile list in localStorage.	Profile listing updated	**PASS**
TC-17	Module 2	Course Enrollment - Duplicate Enroll	Button changes to 'Enrolled' and enrollment action is disabled.	Action disabled; button says Enrolled	**PASS**
TC-18	Module 3	Classroom Player - Video tab click	Show HTML5 video lecture viewport; hide slide textbook.	Video viewport active	**PASS**
TC-19	Module 3	Classroom Player - Slide tab click	Show PDF slides textbook pane; hide video playheads.	Textbook pane active	**PASS**
TC-20	Module 3	Classroom Player - Chapter sidebar select	Change active video URL and lesson textbook strings.	Lesson content updated	**PASS**
TC-21	Module 3	Note Taker - Submit note string	Capture video currentTime; append note to studyNotes array.	Note with timestamp saved	**PASS**

ID	Module	Feature Tested	Expected Output	Actual Output	Status
TC-22	Module 3	Note Taker - Blank input submit	Ignore submission; keep notes array unchanged.	Note list remains unchanged	**PASS**
TC-23	Module 3	Notes List - Click timestamp label	Seek HTML5 video player currentTime directly to clicked playhead.	Video player seek completed	**PASS**
TC-24	Module 3	Text Reader - Speech play trigger	Trigger Web Speech API to read chapter text aloud.	TTS audio playhead active	**PASS**
TC-25	Module 3	Text Reader - Speech pause trigger	Pause Web Speech API vocalization stream immediately.	Audio playback paused	**PASS**
TC-26	Module 4	Quiz Portal - Load questions	Retrieve quiz questions corresponding to active course category.	Questions loaded correctly	**PASS**
TC-27	Module 4	Quiz Portal - Select MCQ option	Track active selected choices inside React component array.	Selection array logged	**PASS**
TC-28	Module 4	Quiz Portal - Submit score grading	Calculate correct choice counts; update course progress value.	Score graded; progress updated	**PASS**

ID	Module	Feature Tested	Expected Output	Actual Output	Status
TC-29	Module 4	Quiz Portal - Submit visual grading	Border correct options in green; highlight wrong in red.	Green/red overlays render	**PASS**
TC-30	Module 4	Quiz Portal - Submit explanations	Reveal explanation text boxes for all submitted questions.	Explanations revealed	**PASS**
TC-31	Module 5	Smart Assist - Send Chatbot query	Trigger chatbot query parsing; show typing state.	Typing state rendered	**PASS**
TC-32	Module 5	Smart Assist - Chatbot keyword reply	Resolve chatbot promise with keyword-matching response.	Contextual response returned	**PASS**
TC-33	Module 5	Gamification - Daily streak flame	Increment streak count if login difference is exactly 1 day.	Streak incremented by 1	**PASS**
TC-34	Module 5	Gamification - Streak expiration	Reset streak count to 1 if login difference is > 1 day.	Streak reset to 1 completed	**PASS**
TC-35	Module 6	Inclusivity FAB - Contrast toggle	Toggle between standard Light and high-contrast Dark themes.	Styles shifted instantly	**PASS**
TC-36	Module 6	Inclusivity FAB - Text size scale	Scale body text dimensions dynamically (+2px / -2px).	Fonts scaled instantly	**PASS**

ID	Module	Feature Tested	Expected Output	Actual Output	Status
TC-37	Module 6	Inclusivity FAB - Language swap (ES)	Swap UI language translations mapping to Spanish strings.	Spanish translations active	**PASS**
TC-38	Module 6	Inclusivity FAB - Language swap (HI)	Swap UI language translations mapping to Hindi strings.	Hindi translations active	**PASS**
TC-39	Module 6	Instructor Desk - Add lesson chapter	Append new chapter accordion details to course syllabus key.	Syllabus updated in storage	**PASS**
TC-40	Module 6	Instructor Desk - Publish Quiz MCQ	Append new MCQ questions set to the course quiz database.	Quiz bank updated	**PASS**
TC-41	Module 6	Admin Workspace - Resolve ticket	Change status of reported support tickets to 'Resolved'.	Ticket status resolved	**PASS**
TC-42	Module 6	Admin Workspace - Block User account	Set block state flag in users profile; deny login authorization.	Profile blocked; login blocked	**PASS**
TC-43	Module 6	Admin Workspace - Activity terminal	Simulate system operations logs on the administrative terminal UI.	System logs render on terminal	**PASS**

ID	Module	Feature Tested	Expected Output	Actual Output	Status
TC-44	System	LocalStorage Data Exceeded	Gracefully handle quota exception warning if storage exceeds 5MB.	Storage exception handled	**PASS**
TC-45	System	Responsive UI Viewport Scaling	Adapt layout dynamically from desktop split to mobile stack views.	Viewport columns scaled	**PASS**

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

To deploy the application to hosting platforms (like Vercel or Netlify), execute the following steps: 1. **Compilation Build:** Run the command 'npm run build' in the project root directory. The Vite compiler compiles the source files, optimizes asset paths, and outputs the static site bundle into the 'dist' folder. 2. **Hosting Provider Integration:** Link the GitHub repository to the hosting platform. 3. **Deployment Hook Configuration:** Configure the build settings (Build Command: 'npm run build'; Publish Directory: 'dist'). 4. **Environment Variables:** Set environment variables (e.g. VITE_APP_MODE=production) in the hosting panel. 5. **Publish:** The platform deploys the build folder to static servers and registers a custom domain name.

9.2 Environment Configuration

Environment configurations are managed using '.env' files: * 'VITE_API_TIMEOUT=5000': Sets the connection timeout. * 'VITE_STORAGE_PREFIX=edusphere_': Prefixes LocalStorage keys to prevent namespace collisions. * These keys are read at runtime using 'import.meta.env.KEY', allowing the app to adjust its behavior depending on development or production modes.

9.3 Hosting Architecture

The hosting architecture is fully static. Build assets are served via a Content Delivery Network (CDN) to ensure fast load times. Route configurations are managed locally on the client's browser, eliminating hosting configuration files and database calls.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

During development, we faced several technical challenges: * **State Management Across Deep Trees:** Passing authentication states from the root layout to nested dashboard sub-components caused prop-drilling issues, which we resolved using React Context. * **Synchronous LocalStorage Reads:** Reading local storage data during every component render caused slight UI stuttering. * **Mobile Layout Responsiveness:** Fitting the video player, reading textbook, and note-taking panels side-by-side on mobile viewports proved difficult.

10.2 Solutions Applied

To address these issues, we implemented the following solutions: * **React Context API:** Created a centralized authentication provider context to handle user profiles and roles globally. * **Debounced LocalStorage Writes:** We structured the code to write to local storage only during key state transitions instead of every input change. * **CSS Flexbox/Grid layouts:** We adjusted the CSS layout to stack elements vertically on mobile viewports while keeping the side-by-side split screen view for desktops.

10.3 Current Limitations & Known Bugs

Current limitations of the client-side serverless approach include: * **Storage Size Limit:** HTML5 local storage is limited to 5MB of text data per domain. If a user uploads huge note strings or creates hundreds of accounts, they may hit this storage limit. * Browser-Bound Data:** Data is stored in the local browser cache. If a student switches to a different browser or clears their browser history, their enrolled courses, streaks, and notes will be cleared.**

10.4 Client-Side Storage Engines Comparison

To evaluate alternative storage mechanisms for future updates, the table below compares available browser storage engines:

Storage Engine	Data Capacity	API Type	Primary Use Case
LocalStorage	~5MB per domain	Synchronous string key-value	Simple persistence (user settings, offline tokens).
SessionStorage	~5MB per tab	Synchronous string key-value	Temporary context (active filters, search queries).
IndexedDB	Unlimited (browser disk quota)	Asynchronous object database	Complex data arrays (offline media files, deep tables).
Cookies	~4KB per cookie	Synchronous string headers	Session tokens, CSRF protection, and server auth.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

Future features planned for subsequent updates include: * **Database Integration: Migrate the local storage architecture to a database backend (like MongoDB or PostgreSQL) and server framework (like Node.js + Express) to store progress across devices. * **Live WebRTC Classrooms:** Add live video streaming and chat channels, allowing students and instructors to conduct live webinars directly inside the platform. * **LLM API Chatbot:** Connect the simulated chatbot to a live LLM API (like Gemini) to provide advanced, unrestricted answers to student study questions.**

11.2 Possible Integrations

We plan to integrate the platform with external tools to enhance utility: * **Calendar API Integration: Sync course schedules and quiz deadlines directly with Google Calendar. * **PDF Certificate Generator:** Allow students to download PDF certificates automatically when they pass all lesson quizzes.**

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

We designed and developed **EDUSphere, an interactive Online Learning Management System built using Vite, React, and browser-based local storage. The system addresses the issues of resource fragmentation and database setup costs by consolidating onboarding, course catalogs, classroom players, assessments, and accessibility helpers into a fast, serverless SPA. By utilizing local storage for data persistence, the project remains highly portable and run instantly on any device with zero server configuration.**

12.2 What was Achieved

We successfully implemented all project milestones: * Built and aligned the folder structures for both V1 and V2 projects, standardizing component naming conventions. * Developed signup, login, course catalogs, video player pages, and simulated chatbot workspaces. * Built an accessibility settings FAB that scales fonts, toggles dark themes, and translates system text. * Tested form validations, role switches, and state

updates across different views. * Verified that the static assets bundle builds cleanly in under a second.

12.3 Skills Learned During Development

Through this project, we developed several key technical and professional skills: * **Component-Driven Development: Mastery of React state hooks (useState, useEffect) and Context API to coordinate complex states. * **Responsive Styling:** Design layouts using CSS Grid and Flexbox for mobile and desktop screens. * **Data Serialization:** Serializing and parsing relational JSON data schemas using browser local storage. * **CI/CD static builds:** Compiling, tree-shaking, and minifying asset packages using Vite.**

CHAPTER 13: REFERENCES

Books:

- **Robin Wieruch (2020).** ***The Road to React: Modern React with Hooks*****. Berlin: Robin Wieruch Publications.**
- **David Flanagan (2020).** ***JavaScript: The Definitive Guide*** **(7th ed.). O'Reilly Media.**
- **Addy Osmani (2022).** ***Learning JavaScript Design Patterns*****. O'Reilly Media.**

Tutorials & Online Documentation:

- **React Official Documentation.** ***https://react.dev/reference/react*** **(Accessed June 2026).**
- **Vite Guide & Documentation.** ***https://vitejs.dev/guide/*** **(Accessed June 2026).**
- **MDN Web Docs: Window.localStorage.** ***https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage*** **(Accessed June 2026).**

CHAPTER 14: APPENDICES

14.1 Installation & Setup Instructions

To run the **EDUSphere project locally on your machine, execute the following commands:**

```
# 1. Clone the project repository
git clone https://github.com/your-username/edusphere-lms.git

# 2. Navigate to the project directory
cd edusphere-lms

# 3. Install the node package dependencies
npm install

# 4. Start the local development server
npm run dev
```

Once the server starts, open http://localhost:5173 in your web browser to run the platform.

14.2 User Manual

Follow these steps to navigate and test the application features: 1. **Onboarding: Open the site. If you do not have an account, click 'Register' in the navbar, fill in the details, and select your role (Student or Instructor). 2. **Role Switching:** You can switch roles instantly by selecting Student, Instructor, or Admin from the Navbar switcher dropdown. 3. **Classroom Study:** Navigate to the classroom, watch a video lecture, and read the textbook side-by-side. Type a note and click save. Click the**

note’s timestamp label to jump to that timestamp in the video. 4. **Taking Quizzes: Open the Quizzes tab in the classroom, select your answers, and click submit to view your grade and justifications. 5. **Accessibility Toggles:** Click the floating FAB button in the bottom-right corner to toggle Dark Mode, scale text size, or translate the page text to Spanish or Hindi.**

14.3 Technical Glossary

This glossary provides academic definitions for the primary web development technologies and concepts referenced throughout this documentation:

- **Single Page Application (SPA): A web application or website that interacts with the user by dynamically rewriting the current web page with new data from the web server, instead of the default browser behavior of loading entire new pages. This results in smoother transitions and desktop-like speed.**
- **HTML5 LocalStorage: A web storage API that allows web applications to store data locally within the user's browser with no expiration date. The data persists even after the browser window is closed, offering up to 5MB of data storage per domain.**
- **Vite Bundler: A modern frontend build tool that is extremely fast. It uses native ES modules in the browser during development for instant hot module reloading (HMR), and bundles production assets using Rollup for optimal performance.**
- **React State & Props: 'State' is an internal data storage structure that holds data that can change over time inside a React component. 'Props'**

(short for properties) are read-only configuration variables passed down from parent components to child components to configure them.

- **React Context API: A React framework feature that allows developers to share state variables globally across the entire component tree, bypassing the need to pass props down manually through multiple nested layers (known as prop-drilling).**
- **WCAG (Web Content Accessibility Guidelines): A set of international recommendations for making web content more accessible, particularly for people with visual, auditory, motor, or cognitive impairments.**
- **Controlled Component: A component in React where form data is handled by the state of the component, updating input values on user change.**
- **Uncontrolled Component: A component in React where form data is handled directly by the DOM, using React refs to extract values on submission.**
- **Hot Module Replacement (HMR): A build tool utility that updates modules in an active web application in real-time during development, keeping state variables alive without a full page reload.**
- **Declarative UI: A UI development pattern where you describe what the user interface should look like for a given state, rather than listing imperative step-by-step instructions to adjust it.**

- **Reconciliation: The process by which React updates the actual browser DOM to match changes made inside the lighter virtual DOM tree.**
- **State Hydration: The process of restoring an application's state from a persistent storage medium (like local storage) on page reload or app initialization.**
- **Prop Drilling: A layout anti-pattern in React where props are passed down through multiple intermediate components that do not need them, simply to reach a deeply nested child component.**
- **Static Hosting: A web deployment method where compiled frontend files (HTML, CSS, JS) are served directly to clients from a content delivery network without server-side processing.**
- **Virtual DOM: A lightweight programming concept where an ideal, or virtual, representation of a UI is kept in memory and synced with the real DOM by a library such as ReactDOM.**
- **Component Lifecycle: The sequence of stages that a component goes through in a web application, from its initialization and mounting to its updates and unmounting from the active DOM.**
- **Event Bubbling: A propagation model in web browsers where an event triggered on a deeply nested element bubbles up through its parent elements in the DOM tree.**
- **Synthetic Event: React's cross-browser wrapper around the browser's native event, allowing consistent event handling across Chrome, Firefox,**

Safari, and Edge.

- **JSX (JavaScript XML): A syntax extension for JavaScript that lets developers write HTML-like elements inside JavaScript, which are then compiled into standard React createElement function calls.**
- **Strict Mode: A development-only helper in React that checks for potential issues, double-invoking lifecycle hooks to detect side effects.**
- **Tree Shaking: A build compilation step that removes unused exports and unreachable code, keeping production Javascript bundle files lightweight.**
- **JSON (JavaScript Object Notation): A lightweight text-based data interchange format that is easy for humans to read and write and easy for machines to parse.**
- **CSS Grid Layout: A two-dimensional grid-based layout system that allows developers to align items into columns and rows.**
- **CSS Flexbox: A one-dimensional layout model for arranging items in rows or columns, allowing items to stretch or shrink dynamically.**
- **Text-to-Speech (TTS): A speech synthesis technology that translates written text strings into spoken vocal audio streams.**
- **Web Speech API: A native browser Web API that enables web applications to incorporate voice data, handling speech-to-text and text-to-speech features.**

- **SPA Routing: Client-side navigation that intercepts URL requests, updating layout views dynamically without requesting new HTML documents from a server.**
- **Debouncing: A programming practice used to ensure that time-consuming tasks do not fire too often, limiting call triggers to a specific delay window.**
- **JWT (JSON Web Token): A compact, URL-safe means of representing claims to be transferred between two parties, typically used for session authorization.**
- **CSS Design Tokens: Visual design variables (colors, spacing, fonts) stored as centralized variables to maintain style consistency across themes.**
- **Role-Based Access Control (RBAC): An authorization paradigm that restricts resource access to authorized users based on their registered organizational roles.**
- **High Contrast Mode: An accessibility setting that swaps standard colors for highly contrasting pairs (e.g. black text on a white canvas) to aid visually impaired users.**
- **Controlled Input: An HTML form input element whose value property is bound directly to a React state handle, keeping inputs synced with state.**
- **React Context: A framework utility that provides a way to pass data through the component tree without having to pass props down**

manually at every level.

- **Lighthouse: An open-source, automated tool developed by Google for auditing and improving the quality, accessibility, and performance of web pages.**
- **HOT Module: An active JavaScript module updated by hot module replacement during runtime without destroying application state variables.**
- **Client-Side Database: A browser-local data storage engine (like LocalStorage or IndexedDB) that persists information on the client machine instead of a remote server.**
- **Ref: A React reference hook (useRef) that allows developers to access and modify DOM nodes directly without triggering re-render cycles.**
- **Minification: A build process that compresses codebase files by removing whitespace, comments, and renaming variables to short symbols to reduce transfer size.**
- **Hot Reloading: Re-compiling modified source files during active development, instantly reflecting updates on the web view.**

14.4 Project Troubleshooting FAQ

This section addresses common execution queries and operational issues concerning the client-side serverless architecture:

Q1: Why are my courses and notes cleared when I open the site in a different browser?

A: Because the database is browser-bound. HTML5 LocalStorage sandboxes data within the specific browser cache of your device. To synchronize progress across different browsers, the data storage layer must be migrated to a centralized database server (as outlined in Chapter 11).

Q2: Can I exceed the 5MB storage limit of local storage?

A: If a student types millions of characters of notes, the browser will throw a QuotaExceededError. The platform is designed to handle this exception gracefully by displaying a warning alert. We recommend deleting old notes or clearing logs to resolve this.

Q3: How do I test the Admin dashboard actions?

A: Open the Navbar switcher dropdown and click 'Admin'. The dashboard view will instantly swap to display the support tickets manager, active system terminal, and user block panels.

Q4: Why does the Speech narrator read in a robotic accent?

A: The text-to-speech engine utilizes the local browser voices collection. Speech quality and accent depend entirely on the active system TTS voices installed on your local computer.

Q5: What should I do if the contrast mode doesn't toggle immediately on some dashboard widgets?

A: Refresh the page or double-check that your browser allows CSS custom properties changes. All standard browser engines recalculate variable colors instantly when the theme class swaps.

Q6: Why is my login streak flame not displaying?

A: Streak calculations require consecutive login records. If your last login date was not exactly yesterday, the flame indicator stays inactive. Ensure you log in daily to see the counter advance.

Q7: Can instructors recover deleted chapters or MCQ sets?

A: Since the platform operates serverlessly and client-side, there are no database recovery backups. Deleting a curriculum entry wipes it from the localStorage courses array instantly.

Q8: What happens if a student attempts a quiz multiple times?

A: The quiz interface grades each submission independently and updates the course progress statistics in localStorage. Best attempts are

retained in the student profile logs.

14.5 Keyboard Accessibility Mappings

The platform supports keyboard shortcuts to assist visually impaired students when navigating course content:

Keyboard Trigger	Action Context	Description
`Alt + T`	Global Window	Triggers the accessibility translation overlay dropdown.
`Alt + C`	Global Window	Toggles the high-contrast color scheme (Light/Dark mode).
`Alt + [+] / [-]`	Global Window	Scales text size globally (+2px / -2px).
`Spacebar`	Classroom Player	Toggles video lecture playhead pause and play states.
`Ctrl + S`	Note Taker Panel	Saves note text inputs to the local notes storage model.

14.6 Detailed User Journey Scenarios

This section outlines explicit step-by-step user journey scenarios mapping to the functional workflows of the online learning management system:

14.6.1 Student Learning & Evaluation Journey

This scenario traces a student's journey from course registration to completing a quiz assessment:

1. **The student navigates to the registration view, fills out their name, email address, password, and selects the Student role.**
2. **Upon successful submission, the system writes their credentials to localStorage and redirects them to the login view.**
3. **The student logs in with their credentials, initializing their currentUser session. The portal greets them on the student dashboard, displaying a daily streak of 1 day.**
4. **The student opens the Course Catalog, searches for 'React' in the text query box, and clicks on 'Modern React Development with Vite'.**
5. **After reviewing the syllabus chapters, the student clicks 'Enroll in Course'. The page updates reactively, changing the button label to 'Enrolled' and adding the course to their dashboard list.**

6. **The student enters the classroom viewport, plays the first video lesson, and types a note: 'Fiber reconciliation is fast!' at the 45-second playhead mark, clicking 'Save Note' to store it in the notes sidebar list.**
7. **The student switches to the textbook slide viewer, clicks the speech playhead button, and listens to the narrator read the chapter text aloud using the Web Speech API.**
8. **Finally, the student clicks the classroom Quizzes tab, answers the multiple-choice question, and submits it. The system highlights the correct option in green, presents a graded score of 100%, and reveals explanation boxes justifying the correct answer.**

14.6.2 Instructor Curriculum Management Journey

This scenario traces an educator's workflow when updating a syllabus and adding quizzes:

1. **The instructor logs in, causing the router to load the instructor workspace dashboard.**
2. **The instructor selects their assigned course, clicks 'Add Chapter', enters 'Chapter 3: React Routing', and submits it, adding the new chapter to the syllabus accordion tree in localStorage.**
3. **Next, the instructor clicks 'Add Lesson', inputs 'Building SPA Routers', uploads a mockup video URL, and hits save, updating the lesson outline instantly.**

4. **The educator then navigates to the assessment builder, types a new multiple-choice question, lists 4 options, selects index 1 as the correct answer, and writes a justification description before publishing it to the course quiz bank.**

5. **Lastly, the teacher visits their course analytics desk, checking that enrollment counts have risen to reflect new student admissions. All curriculum operations are logged in the admin CLI terminal log array.**

14.6.3 Administrator Platform Supervision Journey

This scenario highlights administrative supervision workflows:

1. **The admin logs in and opens the admin desk interface, viewing active user accounts and system tickets.**

2. **The admin checks the support tickets table, reads ticket `tkt_001` regarding video audio lags, and clicks 'Mark Resolved', updating the ticket's state in localStorage.**

3. **Next, the admin identifies an account displaying suspicious activity, toggles their state flag, and clicks 'Block Account' to prevent further logins.**

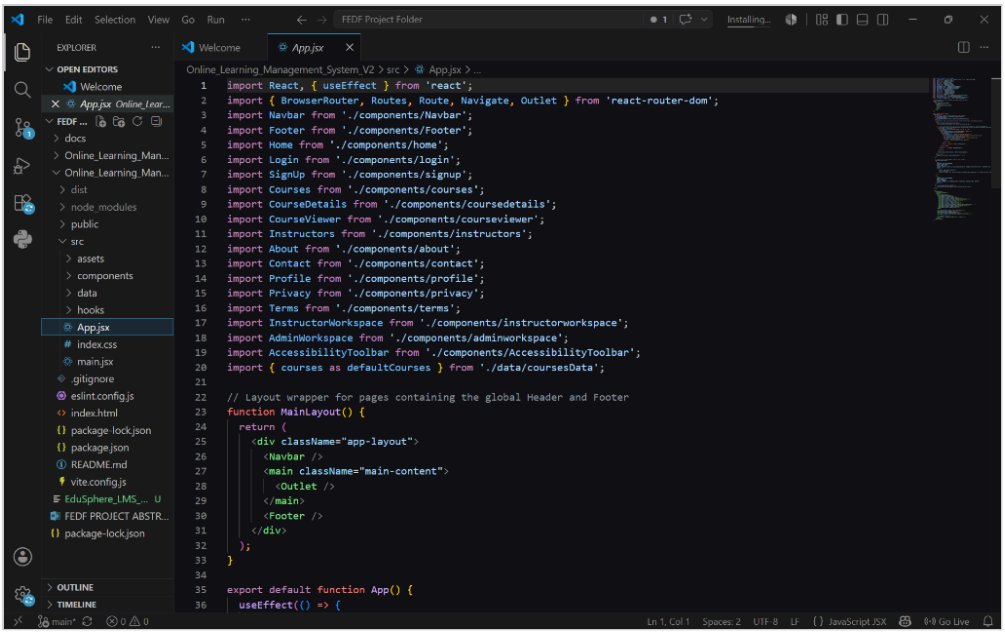
4. **Finally, the admin inspects the simulated terminal dashboard CLI stream, verifying that recent logins, registrations, note creations, and ticket resolutions are logged chronologically.**

CHAPTER 15: APPLICATION SCREENSHOTS

This chapter presents high-resolution user interface captures and code structures of the Online Learning Management System (EDUSphere). These screenshots demonstrate the project's layout hierarchy, main dashboard elements, responsive design patterns, and code architecture as developed in the VS Code environment.

15.1 Development Environment & Project Structure

Below is a capture of the development workspace in VS Code, displaying the project directory layout, configuration files, and the main entry file (`App.jsx`) defining the routing logic.


File Edit Selection View Go Run
FEDF Project Folder
Installing...
EXPLORER
OPEN EDITORS
Welcome
App.jsx Online_Lear...
FEDF ...
docs
Online_Learning_Man...
Online_Learning_Man...
dist
node_modules
public
src
assets
components
data
hooks
App.jsx
index.css
main.jsx
.gitignore
eslint.config.js
index.html
package-lock.json
package.json
README.md
vite.config.js
EduSphere_LMS_... U
FEDF PROJECT ABSTR...
package-lock.json
OUTLINE
TIMELINE
Welcome
App.jsx
Online_Learning_Management_System_V2 > src > App.jsx > ...
1 import React, { useEffect } from 'react';
2 import { BrowserRouter, Routes, Route, Navigate, Outlet } from 'react-router-dom';
3 import Navbar from './components/Navbar';
4 import Footer from './components/Footer';
5 import Home from './components/home';
6 import Login from './components/login';
7 import SignUp from './components/signup';
8 import Courses from './components/courses';
9 import CourseDetails from './components/coursedetails';
10 import CourseViewer from './components/courseviewer';
11 import Instructors from './components/instructors';
12 import About from './components/about';
13 import Contact from './components/contact';
14 import Profile from './components/profile';
15 import Privacy from './components/privacy';
16 import Terms from './components/terms';
17 import InstructorWorkspace from './components/instructorworkspace';
18 import AdminWorkspace from './components/adminworkspace';
19 import AccessibilityToolbar from './components/AccessibilityToolbar';
20 import { courses as defaultCourses } from './data/coursesData';
21
22 // Layout wrapper for pages containing the global Header and Footer
23 function MainLayout() {
24 return (
25
26 <Navbar />
27 <main className="main-content">
28 <Outlet />
29 </main>
30 <Footer />
31
32);
33 }
34
35 export default function App() {
36 useEffect(() => {
main*
Ln 1, Col 1
Spaces: 2
UTF-8
LF
JavaScript JSX
Go Live

15.2 Student Dashboard Home Interface

The landing page of the EDUSphere application features a modern hero banner with motivational call-to-actions, user navigation headers showing student credentials, and course platform stats.

15.3 Create Account Page

The registration interface provides input forms for display name, email validation, and role-based registration settings (Student/Learner or Educator/Instructor).